

**Session 1: Procedural vs Object Thinking**  
**Topics: Limitations of C (global data, no encapsulation). Why OOP solves these. Example: Compare “Task List” written in C vs OOP style.**

**Tasks**

- 1) Write a simple C program tasklist\_c.c that allows you to add up to 5 tasks (as strings) to a global array and print all tasks using a for loop.**

```
#include <iostream>
#include <string>
using namespace std;

string tasks[5];
int taskCount = 0;

int main() {
    cout << "Enter up to 5 tasks:\n";

    for (int i = 0; i < 5; i++) {
        cout << "Enter task " << i + 1 << ": ";
        getline(cin, tasks[i]);
        taskCount++;
    }

    cout << "\n--- Task List ---\n";

    for (int i = 0; i < taskCount; i++) {
        cout << i + 1 << ". " << tasks[i] << endl;
    }

    return 0;
}
```

Output:

```
PROBLEMS  OUTPUT  PORTS
▼ TERMINAL

>> .\demo27
Enter up to 5 tasks:
Enter task 1: complete the c++ concep
Enter task 2: study the evry topic
Enter task 3: inislition task
Enter task 4: create the project
Enter task 5: submit the project

--- Task List ---
1. complete the c++ concep
2. study the evry topic
3. inislition task
4. create the project
5. submit the project
PS C:\Users\Admin\OneDrive\Desktop> .\demo27
```

2) Modify your tasklist\_c.c to add a function markTaskDone(int index) that sets the selected task to 'DONE' in the array, then print the updated list.<br><br><em><strong>Hint:</strong> Use a separate status array or append ' - DONE' to the task string.</em>

**Code-**

```
#include <iostream>

#include <string>

using namespace std;

string tasks[5];
string status[5];

int taskCount = 0;

void markTaskDone(int index) {
    if (index >= 0 && index < taskCount) {
        status[index] = "DONE";
    } else {
        cout << "Invalid task number!" << endl;
    }
}

int main() {

    cout << "Enter 5 tasks:\n";
```

```
for (int i = 0; i < 5; i++) {  
    cout << "Enter task " << i + 1 << ": ";  
    getline(cin, tasks[i]);  
  
    status[i] = "Pending";  
    taskCount++;  
}
```

```
cout << "\n--- Task List ---\n";
```

```
for (int i = 0; i < taskCount; i++) {  
    cout << i + 1 << ". " << tasks[i]  
        << " - " << status[i] << endl;  
}
```

```
int index;  
cout << "\nEnter task number to mark as DONE: ";  
cin >> index;
```

```
markTaskDone(index - 1);
```

```
cout << "\n--- Updated Task List ---\n";

for (int i = 0; i < taskCount; i++) {

    cout << i + 1 << ". " << tasks[i]

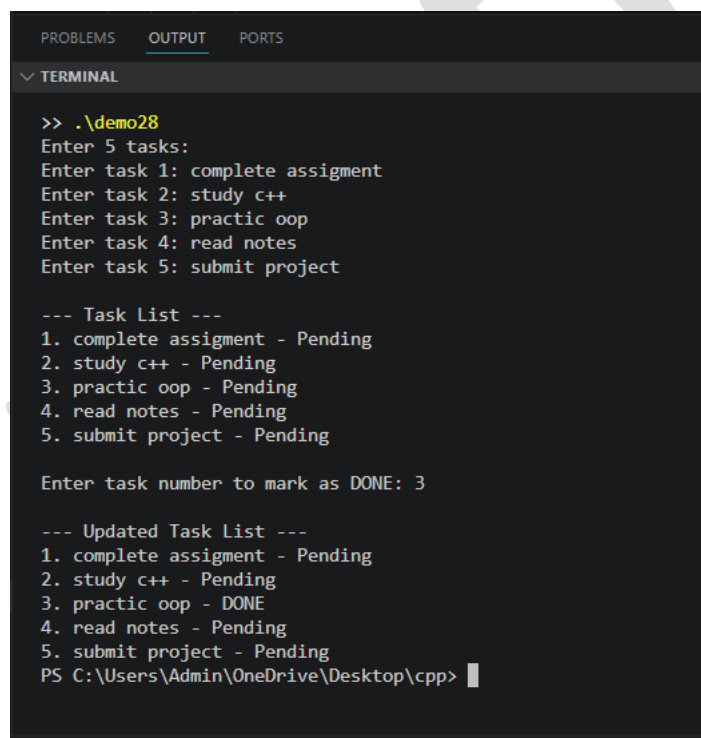
        << " - " << status[i] << endl;

}

return 0;

}
```

Output:



```
PROBLEMS  OUTPUT  PORTS
▼ TERMINAL

>> .\demo28
Enter 5 tasks:
Enter task 1: complete assignment
Enter task 2: study c++
Enter task 3: practic oop
Enter task 4: read notes
Enter task 5: submit project

--- Task List ---
1. complete assignment - Pending
2. study c++ - Pending
3. practic oop - Pending
4. read notes - Pending
5. submit project - Pending

Enter task number to mark as DONE: 3

--- Updated Task List ---
1. complete assignment - Pending
2. study c++ - Pending
3. practic oop - DONE
4. read notes - Pending
5. submit project - Pending
PS C:\Users\Admin\OneDrive\Desktop\cpp>
```

- 3) Create a Task class in your favorite OOP language (Java, Python, or C++) with properties: title and isDone. Add a method markDone() that sets isDone to true, and a method display() that prints the task with its status.**

```
#include <iostream>
#include <string>
using namespace std;

class Task {
private:
    string title;
    bool isDone;

public:
    Task(string taskTitle) {
        title = taskTitle;
        isDone = false;
    }

    void markDone() {
        isDone = true;
    }

    void display() {
        cout << "Task: " << title << endl;

        if (isDone) {
            cout << "Status: DONE" << endl;
        } else {
            cout << "Status: PENDING" << endl;
        }
    }
};

int main() {

    Task task1("Complete C++ Assignment");
```

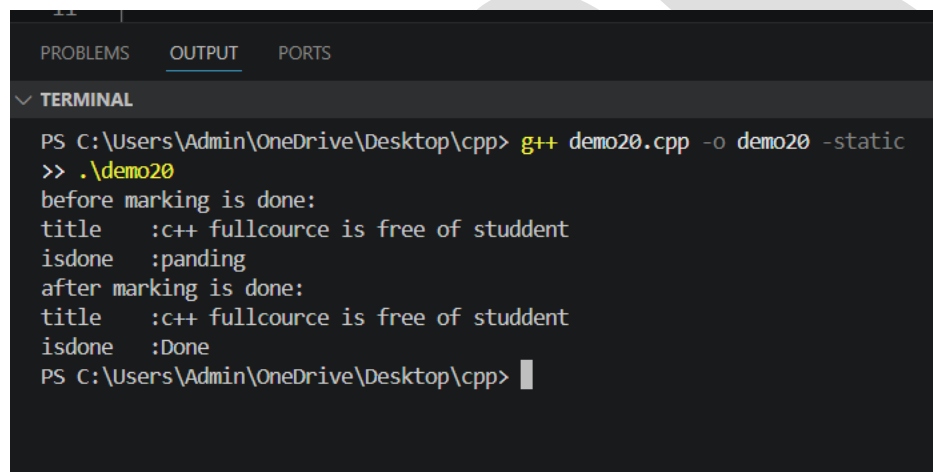
```
cout << "--- Before Marking Done ---" << endl;
task1.display();

task1.markDone();

cout << "\n--- After Marking Done ---" << endl;
task1.display();

return 0;
}
```

Output:



```
PROBLEMS  OUTPUT  PORTS
✓ TERMINAL
PS C:\Users\Admin\OneDrive\Desktop\cpp> g++ demo20.cpp -o demo20 -static
>> .\demo20
before marking is done:
title    :c++ fullcourse is free of student
isdone   :panding
after marking is done:
title    :c++ fullcourse is free of student
isdone   :Done
PS C:\Users\Admin\OneDrive\Desktop\cpp> |
```

- 4) Build a simple TaskList class/object that stores multiple Task objects and provides addTask(title), markTaskDone(index), and showTasks() methods. Demonstrate adding 3 tasks, marking one as done, and displaying all tasks with their statuses.

**Code-**

```
#include <iostream>

#include <vector>

#include <string>

using namespace std;

class Task {
private:
    string title;
    bool isDone;
public:

    Task(string t) {
        title = t;
        isDone = false;
    }

    void markDone() {
        isDone = true;
    }
}
```



```
void display() {  
    cout << "Task: " << title  
        << " | Status: " << (isDone ? "Done" : "Pending") << endl;  
}  
};
```

```
class TaskList {  
private:  
    vector<Task> tasks;  
  
public:  
    void addTask(string title) {  
        tasks.push_back(Task(title));  
    }  
    void markTaskDone(int index) {  
        if (index >= 0 && index < tasks.size()) {  
            tasks[index].markDone();  
        } else {  
            cout << "Invalid task index!" << endl;  
        }  
    }  
};
```

```
void showTasks() {  
    cout << "\nTask List:\n";
```

```
        for (int i = 0; i < tasks.size(); i++) {  
            cout << i + 1 << ". ";  
            tasks[i].display();  
        }  
    }  
};  
  
int main() {  
    TaskList myTasks;  
  
    myTasks.addTask("Complete C++ Assignment");  
    myTasks.addTask("Study OOP Concepts");  
    myTasks.addTask("Practice Coding");  
  
    myTasks.markTaskDone(1);  
  
    myTasks.showTasks();  
  
    return 0;  
}
```

Output:

```
PS C:\Users\Admin\OneDrive\Desktop\cpp> g++ demo21.cpp -o demo21 -static  
>> .\demo21  
  
Task List:  
1. Task: Complete C++ Assignment | Status: Pending  
2. Task: Study OOP Concepts | Status: Done  
3. Task: Practice Coding | Status: Pending  
PS C:\Users\Admin\OneDrive\Desktop\cpp> █
```

**5) Compare your procedural C version and OOP TaskList version: List 3 problems you faced in the C version that were solved by using classes and methods in the OOP version.**

**Procedural C Version**

Uses separate arrays for task data and status.

Functions work separately on global data.

Adding more features makes the code harder to manage.

**OOP TaskList Version**

Uses a Task class to keep task data and behavior together.

Methods like markDone() directly manage the task object.

Classes make the code easier to extend and maintain.

TOPS